

Profiling Report

Face Verification Pipeline — v1.0-final
Course: MSML/MSAI 605 | Milestone 4 | 2025

1. Measurement Environment (CPU Baseline — Required)

All profiling results were collected on CPU hardware. The profiling script (`scripts/profile_latency.py`) reports hardware details at runtime via `platform.platform()` and `os.cpu_count()`. Run the script to populate actual hardware values. GPU comparison is optional supplemental evidence.

Property	Measured Value
Platform	macOS 15.3 (arm64)
Processor	Apple Silicon (arm)
CPU cores	10
Python	3.9.6
PyTorch	2.2.2
TensorFlow	2.20.0
facenet-pytorch	>=2.5.3 (PyTorch backend used)
CUDA available	False (CPU-only baseline)
Model backend	pytorch (FaceNet InceptionResnetV1, VGGFace2)

2. Methodology

- **Input:** Synthetic random 160x160 RGB images with fixed NumPy seed — consistent across runs.
- **Warm-up:** 3 warm-up iterations before measurement to avoid cold-start bias.
- **Timing:** Python `time.perf_counter()` with nanosecond resolution.
- **Single-pair repetitions:** 30 repetitions; statistics reported as mean, median, std, p95.
- **Batch repetitions:** 10 repetitions per batch size.
- **Stage isolation:** Preprocessing, embedding, and scoring each timed with bracketed `perf_counter()` calls.
- **Reproducibility:** Run: `python scripts/profile_latency.py --config configs/m3.yaml --output outputs/profiling/results.json`

3. Per-Stage Latency Breakdown (Single Pair, CPU Baseline)

The table below shows measured values from the CPU baseline run (macOS 15.3, Apple Silicon arm64, 10-core CPU, n=30 repetitions). Full raw data is in `outputs/profiling/results.json`.

Stage	Mean (ms)	Std (ms)	P95 (ms)	% of Total
Preprocessing (x2 images)	0.364	0.107	~0.57	0.8%
Embedding extraction (x2)	43.996	13.367	~64.0	99.1%
Cosine similarity scoring	0.053	0.014	~0.08	0.12%
End-to-end total	44.415	13.366	~64.4	100%

Stage breakdown (% of total):

Stage	Measured % of total	Notes
Preprocessing	0.8%	PIL resize + per-image normalization, called twice per pair
Embedding	99.1%	FaceNet InceptionResnetV1 forward pass — overwhelmingly dominant
Scoring	0.12%	Single NumPy dot product over 512-dim vectors, negligible

Interpretation: The embedding extraction stage dominates end-to-end latency. FaceNet InceptionResnetV1 is a deep network with 512-dimensional output; a forward pass is far more expensive than either preprocessing or scoring. Preprocessing is second (PIL resize + normalization per image). Scoring is negligible — a single vectorized NumPy dot product over 512-dimensional vectors completes in microseconds. The highest-leverage optimization is the embedding stage, via GPU acceleration, model quantization, or caching embeddings for known reference images.

4. Batch-Size Sensitivity

The table below shows measured values from the CPU baseline run (macOS 15.3, Apple Silicon arm64, n=10 repetitions per batch size).

Batch Size	Total Mean (ms)	Per-Image (ms)	Throughput (img/s)	vs. Batch 1
1	18.8	18.79	53.2	baseline
2	32.4	16.18	61.8	+16.2%
4	65.2	16.29	61.4	+15.5%
8	159.5	19.93	50.2	-5.6%
16	268.0	16.75	59.7	+12.2%
32	458.8	14.34	69.7	+31.1%

Interpretation: Throughput generally improves with larger batch sizes, reaching 69.7 img/s at batch 32 vs. 53.2 img/s at batch 1 — a 31% gain. Per-image latency drops from 18.79 ms (batch 1) to 14.34 ms (batch 32). The non-monotonic behavior at batch 8 (throughput dip to 50.2 img/s) reflects natural variance in CPU scheduling across the 10 repetitions. The current implementation calls `extract_embedding` once per image in a Python loop; true batched inference (single forward pass over N images) would yield larger and more consistent speedups. For offline batch processing, batch sizes of 16 or 32 are recommended. For latency-sensitive single-pair applications, batching provides no benefit.

5. CPU Baseline Summary

- End-to-end single-pair latency: 44.4 ms mean (std 13.4 ms) on Apple Silicon arm64 CPU.
- Embedding stage dominates at 99.1% of total latency (44.0 ms mean); this is the only stage worth optimizing.
- Preprocessing contributes 0.8% (0.36 ms); scoring is negligible at 0.12% (0.05 ms).
- Throughput at batch 32: 69.7 img/s — a 31% improvement over single-image throughput (53.2 img/s).
- No GPU was available on the test machine (CUDA: False); all results are CPU-only.
- Full raw results are in `outputs/profiling/results.json`.

6. Optional GPU Comparison

GPU profiling was not collected for this submission. The CPU baseline is sufficient to characterize the system's runtime behavior. GPU acceleration can be enabled by ensuring `torch.cuda.is_available()` returns `True`; the FaceNet model will automatically move to the available CUDA device. If GPU results are collected, they should be clearly labeled as supplemental and accompanied by device name and driver version.

7. How to Reproduce

Run the following commands from the project root:

```
pip install -r requirements.txt
python scripts/profile_latency.py --config configs/m3.yaml --n-repeats 30 --output
outputs/profiling/results.json
cat outputs/profiling/results.json
```

All timing values in this report are reproducible with these commands. Results will vary across machines; the output structure (stage breakdown, batch table) is deterministic.